

SIMATS ENGINEERING

Department of Computer Science & Engineering ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment :	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	A.Gopika	Reg.No.:	192521343
Department:	B.Tech IT	Date:	01.08.2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

/ A.Gopika/192521343 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correct ness of Debug ged Solutio n (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria		Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstand the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with	Partially correct solution with limited	Incorrect solution or fails to resolve errors

			minor errors	functionality	
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

1.Debugging Signed Integer Addition Using 2's Complement (8- bit)

Given Numbers

· **+120 = 01111000**

· **+50 = 00110010**

Step 1: Perform Binary Addition

01111000 (+120)

+ 00110010 (+50)

10101010

Step 2: Interpret the Result

· Binary result: **10101010**

· In 8-bit 2's complement, the MSB is **1**, so it represents a negative number.

· Decimal value:

◦ Invert: 01010101

◦ Add 1: 01010110 = **86**

- Therefore, result = **-86**

But the correct mathematical answer is:

$$120 + 50 = 170$$

Since **170 cannot be represented in 8-bit signed 2's complement** (range: **-128 to +127**), the result wraps around to **-86**.

Step 3: Overflow Analysis

Overflow has occurred.

Rule for Signed Overflow:

- Overflow occurs when:
 - Two **positive** numbers produce a **negative** result, or
 - Two **negative** numbers produce a **positive** result. Here:
 - Operand 1 = Positive
 - Operand 2 = Positive
 - Result = Negative

 **Overflow detected.**

Step 4: Solution to Detect Overflow

Use the sign bits:

If (A_sign == B_sign) AND (Result_sign != A_sign)

Overflow = TRUE

Else

Overflow = FALSE

For this example:

- A sign = 0
- B sign = 0
- Result sign = 1

Overflow = TRUE

Alternative Hardware Method

Overflow can also be detected by comparing the carry bits:

Overflow = Carry into MSB XOR Carry out of MSB If they differ, overflow has occurred.

Step 5: Handling Overflow

When overflow is detected:

1. Set an **overflow flag**.
2. Display an "**Arithmetic Overflow**" error or exception.
3. Use a **larger data type** (e.g., 16-bit or 32-bit integers) if larger values are expected.

2.Debugging Ripple Carry Adder (RCA) Performance

Problem

A **Ripple Carry Adder (RCA)** in a real-time processor is causing **slow arithmetic operations** because each carry must propagate through every bit before the final result is produced.

Step 1: Carry Propagation in Ripple Carry

Adder Example: Add two 4-bit numbers

$$A = 1101$$

$$B = 0111$$

Carry propagation:

$$\text{Bit 0: } A_0 + B_0 + C_0 \rightarrow \text{Sum}_0, \text{Carry}_1$$

$$\text{Bit 1: } A_1 + B_1 + \text{Carry}_1 \rightarrow \text{Sum}_1, \text{Carry}_2$$

$$\text{Bit 2: } A_2 + B_2 + \text{Carry}_2 \rightarrow \text{Sum}_2, \text{Carry}_3$$

$$\text{Bit 3: } A_3 + B_3 + \text{Carry}_3 \rightarrow \text{Sum}_3, \text{Carry}_4$$

The carry must travel through **all bits sequentially**.

Bit0 → Bit1 → Bit2 → Bit3

If each full adder takes t ns,
Total Delay = $n \times t$

For a 32-bit RCA:

Delay = $32t$

This long carry chain creates the **performance bottleneck**, especially in real-time systems.

Step 2: Performance Bottleneck

- Carry output of one stage becomes the input of the next stage. ·
- Higher-order bits must wait for previous carries. · As the number of bits increases, delay increases linearly. **Reason:** Sequential carry propagation.

Step 3: Optimized Solution – Carry Look-Ahead Adder (CLA)

A **Carry Look-Ahead Adder (CLA)** computes carry signals in **parallel** instead of waiting for each previous carry.

It uses:

· **Generate:** $G_i = A_i \times B_i$

· **Propagate:** $P_i = A_i \oplus B_i$

Carries are calculated as:

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 +$$

P3P2P1P0C0 All carries are generated **simultaneously**.

Step 4: How CLA Reduces Delay

Ripple Carry Adder

Carry:

Bit0 → Bit1 → Bit2 → Bit3

Delay increases with every bit.

Carry Look-Ahead Adder

Carry Logic

↓

C1 C2 C3 C4 (Generated Together)

↓

Sum Bits

Carries are available immediately, so the sum can be computed much faster.

Comparison

Feature	Ripple Carry Adder	Carry Look Ahead Adder
Carry generation	Sequential	Parallel

Delay	High ($n \times t$)	Low (approximately $\log n$)
Speed	Slow	Fast
Hardware complexity	Simple	More complex

Suitable for	Small circuits	High-speed processors
--------------	----------------	-----------------------

3. Debugging Booth's Algorithm for Signed Multiplication

Problem

A multiplication unit using **Booth's Algorithm** produces incorrect results for **negative operands**. The error may occur during **bit-pair checking, arithmetic shifts, or sign extension**.

Example

Multiply:

Multiplicand (M) = -6 = 1010 (4-bit 2's complement)

Multiplier (Q) = +5 = 0101

Initialize:

A = 0000

Q = 0101

Q-1 = 0

Step-by-Step Execution

Step Q0 Q-1 Operation After Arithmetic Shift

1 10 A = A - M Shift A, Q, Q-1

2 11 No operation Shift

3 01 A = A + M Shift

4 10 A = A - M Shift

After all iterations, the final **AQ** register contains the product.

Correct result:

$-6 \times 5 = -30$

Possible Sources of Error

1. Incorrect Bit-Pair Checking

Booth's rules are:

Q0 Q-1 Operation

00 No operation

11 No operation

01 Add M

10 Subtract M

Error: Performing the wrong operation for a bit pair. **Correction:** Follow the Booth decision table exactly.

2. Incorrect Arithmetic Right Shift

An **arithmetic right shift** must preserve the sign bit (MSB).

Example:

Before shift:

A = 1011

Correct arithmetic shift:

1101

Error: Using a logical shift inserts 0 into the MSB, changing a negative number into a positive value.

Correction: Always perform an **arithmetic right shift** that copies the sign bit.

3. Incorrect Sign Extension

Negative numbers must retain their sign when extended.

Example:

4-bit -6 = 1010

Correct 8-bit:

11111010

Incorrect:

00001010

Error: Zero extension instead of sign extension.

Correction: Replicate the sign bit (MSB) during

extension. **4. Incorrect Two's Complement During**

Subtraction

Since:

$$A - M = A + (2\text{'s complement of } M)$$

Error: Not computing the two's complement correctly.

Correction: Verify the subtraction uses the proper two's complement of the multiplicand.

Corrective Measures

- Use the correct Booth bit-pair rules (00, 01, 10, 11).
- Perform **arithmetic** (not logical) right shifts.
- Preserve the sign bit during every shift.
- Apply proper **sign extension** for negative operands.

Verify two's complement conversion before subtraction.

- Test with positive, negative, and mixed-sign operand combinations.

4. Debugging Restoring Division Algorithm

Problem

An embedded system using the **Restoring Division Algorithm** is slow because it repeatedly performs **restoration operations**, increasing execution time.

Step 1: Workflow of Restoring Division

For each bit of the dividend:

1. Left shift the **A-Q** registers.
2. Subtract the divisor (**M**) from the accumulator (**A**).
3. Check the result:
 - If $A \geq 0$, set quotient bit = **1**.
 - If $A < 0$, **restore** the previous value by adding **M** back to **A**, and set quotient bit = **0**.
4. Repeat until all bits are processed.

Step 2: Where the Inefficiency Occurs

The performance bottleneck occurs when the subtraction result is negative.

$$A = A - M$$

If $A < 0$

$A = A + M \leftarrow$ Restoration (extra operation)

Quotient bit = 0

The **restoration step** is unnecessary overhead because it requires an additional addition whenever the subtraction fails.

Step 3: Optimized Solution – Non-Restoring Division

The **Non-Restoring Division Algorithm** eliminates the immediate restoration step.

Instead:

- If the previous remainder is **positive**, subtract the divisor.
- If the previous remainder is **negative**, add the divisor in the **next iteration**.
- Perform only one final correction if the remainder is negative after all iterations.

Workflow

If $A \geq 0$

$A = A - M$

Else

$A = A + M$

Shift A-Q

Set quotient bit

Repeat

If final $A < 0$

$A = A + M$

Step 4: Performance Improvement

Restoring Division

Shift



Subtract



Negative?



Yes → Restore (Add back)



Next Step

Extra restoration increases the number of arithmetic operations. **Non-Restoring Division**

Shift



Subtract/Add



Continue directly



Final correction (if needed)

No restoration is performed after every negative result.

Comparison

Feature	Restoring Division	Non-Restoring Division	Restoration
---------	--------------------	------------------------	-------------

step	Required	Not required	Arithmetic operations
	More	Fewer	

Execution time	Higher	Lower	
----------------	--------	-------	--

	Hardware complexity	Moderate	Slightly more control logic
Feature	Restoring Division	Non-Restoring Division	
Performance	Slower	Faster	

Advantages of Non-Restoring Division

- Eliminates repeated restoration operations.
 - Reduces the number of additions/subtractions.
 - Improves execution speed.
 - Increases processor efficiency in real-time embedded systems. ·
- Suitable for high-speed arithmetic units.

5.Debugging IEEE 754 Single Precision Floating-Point Addition Problem

A scientific application using **IEEE 754 single-precision (32-bit)** floating-point numbers produces inaccurate results when adding **very small** and **very large** values.

Example:

$$1.0 \times 10^8 + 1.0$$

The expected result is:

100000001

However, the computed result may be:

100000000

The small value is lost due to limited precision.

Step 1: Exponent Alignment

Before addition, both numbers must have the same exponent.

Large Number : 1.0×2^{26}

Small Number : 1.0×2^0

To align exponents, the mantissa of the smaller number is shifted right many times.

Small Mantissa

1.000000...

↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓↓

0.000000000000...

After many shifts, the smaller value becomes too small to affect the result.

Issue: Significant digits of the smaller number are discarded.

Step 2: Normalization

After addition, the result is normalized so that the mantissa has the correct form.

$1.xxxxx \times 2^E$

Normalization may shift the mantissa and adjust the exponent, which can introduce additional precision loss.

Step 3: Rounding Errors

IEEE 754 single precision stores:

- **1 sign bit**
- **8 exponent bits**
- **23 fraction (mantissa) bits**

Since only **23 bits of precision** are available, extra bits are rounded.

This rounding can introduce small numerical errors that accumulate over many operations.

Source of Inaccuracy

- Large exponent difference causes loss of the smaller operand

during alignment.

- Limited **23-bit mantissa** reduces precision.
- Normalization and rounding introduce additional error.

Optimization Techniques

1. Use Double Precision (Recommended)

Double precision (64-bit) provides:

- 1 sign bit
- 11 exponent bits
- **52 mantissa bits**

Benefits:

- Higher precision
- Smaller rounding errors
- Better handling of very large and very small numbers

2. Use Numerically Stable Algorithms

Instead of adding numbers in arbitrary order:

- Add numbers of **similar magnitude first**.
- Add small numbers together before combining them with large numbers.

This reduces precision loss.

3. Use Compensated Summation

Algorithms such as **Kahan Summation** keep track of small rounding errors and add them back in later operations.

This improves the accuracy of repeated floating-point additions.

4. Reduce Repeated Rounding

- Avoid unnecessary intermediate calculations.
- Keep computations at higher precision until the final result.

Comparison

Feature	Single Precision	Double Precision	Total bits
---------	------------------	------------------	------------

	32	64	
--	----	----	--

Mantissa bits	23	52	
---------------	----	----	--

Precision	Lower	Higher	
-----------	-------	--------	--

Rounding error	More	Less	
----------------	------	------	--

Scientific accuracy	Moderate	High	
---------------------	----------	------	--